

17 · Resumen ejecutivo

Documentacion del sistema

Version analizada: **0.1.0**

Commit analizado: **2282a33**

Fecha de generacion: **2026-08-27**

Generado a partir de 17-executive-summary.md — los Markdown son la fuente unica.
No editar este PDF: editar el Markdown y regenerar.

Contenido de este documento

1. Qué es
2. Qué necesidad cubre
3. Quién lo usa
4. Capacidades principales
5. Lo que deliberadamente ****no**** hace
6. Tecnologías
7. Arquitectura resumida
8. Estado actual
9. Fortalezas
10. Riesgos y limitaciones
11. Oportunidades de mejora
12. Próximos pasos recomendados
13. En una frase

Presentación del sistema para dirección, clientes, evaluadores y posibles colaboradores. Sin detalle de implementación: solo lo que hace falta para decidir.

1. Qué es

RootCause macOS Inspector es una aplicación de escritorio y de línea de comandos que observa el estado de seguridad y de recursos de un Mac, detecta lo que ha cambiado respecto a un estado conocido y explica con evidencia por qué algo merece atención.

Está escrito en Rust, funciona sin conexión y no envía datos a ningún servidor. Es software libre bajo licencia Apache 2.0. Versión analizada: **0.1.0**.

Es la edición macOS de una familia de cuatro sensores que comparten idea y arquitectura: Windows, macOS, navegador y móvil.

2. Qué necesidad cubre

macOS incorpora buenas defensas —Gatekeeper, SIP, FileVault, XProtect, permisos de privacidad—, pero **no responde dos preguntas prácticas**:

1. *¿Siguen todas encendidas y al día?* La respuesta está repartida entre seis comandos y tres paneles de Ajustes.
2. *¿Qué ha cambiado desde ayer?* El sistema no guarda esa comparación.

RootCause reúne las respuestas en una vista y añade lo que falta: la comparación contra un estado bueno conocido. Cuando aparece un programa que se ejecuta al arrancar y nadie lo puso, o una aplicación obtiene permiso para leer todo el disco, o el router responde con otra dirección física, el producto lo señala y muestra la prueba.

3. Quién lo usa

Perfil	Para qué
Usuario técnico de macOS	Revisar su propio equipo periódicamente
Analista de seguridad	Capturar evidencia reproducible de un Mac sospechoso
Administrador de una flota pequeña	Verificar que los controles nativos siguen activos
Desarrollador	Diagnosticar consumo; integrar la salida JSON en scripts
Auditor	Obtener un reporte con la evidencia textual de cada comprobación

4. Capacidades principales

Capacidad	Qué aporta
Inventario de arranque automático	Todo lo que se ejecuta al iniciar el equipo o la sesión, clasificado por riesgo
Detección de cambios	Comparación contra un estado bueno conocido en cuatro superficies
Verificación de defensas nativas	Seis controles con la evidencia del comando que respondió
Auditoría de privacidad	Qué aplicación puede grabar la pantalla, leer el teclado o acceder a todo el disco
Antigüedad del antimalware de Apple	Detecta actualizaciones automáticas rotas

Visibilidad de red	Conexiones por proceso, puertos expuestos y equipos vecinos
Heurísticas de comportamiento	Ocho señales de consumo y de actividad sostenida
Evidencia exportable	JSON completo y reporte forense en Markdown
Historial local	Tendencias, incidentes y auditoría de acciones en SQLite

5. Lo que deliberadamente no hace

Es tan importante como lo anterior, y está declarado en el propio producto:

- **No elimina malware ni bloquea por firma.** No es antivirus ni EDR: los complementa.
- **No actúa por su cuenta.** Toda intervención parte de un clic del usuario y queda auditada. Incluso el bloqueo de una dirección IP se entrega como comando para que lo ejecute una persona.
- **No envía datos.** Cero telemetría. La única salida posible es un adaptador de IA opcional, apagado de fábrica, que envía solo el incidente ya resumido.
- **No pide privilegios de administrador.** Cuando un dato requiere permisos que no tiene, usa una vía alternativa o declara la limitación.
- **No se instala como servicio.** Una herramienta que vigila el arranque automático ajeno no añade el suyo en silencio.

6. Tecnologías

Elemento	Detalle
Lenguaje	Rust (edición 2021, mínimo 1.82)
Interfaz	egui / eframe, nativa, sin navegador embebido
Almacenamiento	SQLite embebido, archivo local del usuario
Dependencias directas	10
Plataforma	macOS 13 o superior, Apple Silicon e Intel
Distribución	.app, .dmg y compilación desde el código
Tamaño del código	12 956 líneas en 23 archivos

7. Arquitectura resumida

Un solo binario con dos caras —interfaz gráfica y línea de comandos— sobre un motor común. Toda la información de un instante vive en un único objeto que alimenta a la vez la pantalla, la consola, los exportes y el historial; ninguna capa vuelve a consultar el sistema por su cuenta.

Todo el contacto con macOS pasa por un único módulo, lo que tiene dos consecuencias prácticas: el resto del código se puede probar sin un Mac detrás, y la superficie de interacción con el sistema operativo es auditable de un vistazo.

La interfaz nunca se bloquea: el motor vive en un hilo propio, de modo que una captura lenta no congela la ventana.

8. Estado actual

Indicador	Valor
Versión	0.1.0

Madurez	Producto funcional y completo en su alcance declarado
Tests	112, todos en verde
Análisis estático	Sin advertencias, con la configuración más estricta
Integración continua	Tres flujos: validación, publicación y web del producto
Documentación	39 documentos de producto + 20 de sistema
Deuda técnica registrada	24 hallazgos, ninguno crítico
Vulnerabilidades conocidas del propio código	Ninguna encontrada en este análisis

9. Fortalezas

1. **Honestidad como característica.** El producto declara sus limitaciones en cada salida: qué no pudo leer, qué no puede saber y qué no va a hacer. Un control cuyo estado no se puede determinar se muestra en amarillo, nunca en verde.
2. **Evidencia siempre visible.** Ninguna afirmación aparece sin el dato crudo que la respalda. Es lo que hace utilizable el resultado en una investigación.
3. **Privacidad verificable.** No hay telemetría, y la ausencia se puede comprobar leyendo el código: no existe cliente HTTP en las dependencias.
4. **Calidad de ingeniería medible.** 112 pruebas, análisis estático sin excepciones, validación de dos ediciones distintas en cada cambio y prueba de humo real en la integración continua.
5. **Decisiones documentadas en el propio código.** Cada elección no evidente lleva su justificación al lado, lo que reduce drásticamente el conocimiento tácito necesario para mantenerlo.
6. **Sin dependencias innecesarias.** Diez dependencias directas para un producto de este alcance, y ninguna añadida por una funcionalidad opcional.

10. Riesgos y limitaciones

Riesgo	Naturaleza	Mitigación disponible
La primera foto se toma por buena	Estructural del enfoque	Revisión inicial de las entradas de riesgo alto (propuesta, no implementada)
Aceptar una baseline no se puede deshacer	Producto	Versionado de baselines (propuesta)
Binarios sin firmar ni notarizar	Distribución	Compilar desde el código, que es la vía recomendada
Sin Acceso total al disco no hay auditoría de privacidad	Modelo de permisos de macOS	El producto lo declara y explica cómo concederlo
Sin privilegios elevados, la vista de red es parcial	Modelo de permisos	Declarado en cada ejecución
Cinco opciones de configuración sin efecto	Deuda técnica	Implementarlas o retirarlas
Dependencias sin vigilancia automática de vulnerabilidades	Proceso	Añadir un paso de auditoría a la integración continua
No detecta lo que ocurre entre dos capturas	Estructural del muestreo	Reducir el intervalo, con coste de recursos

Ninguno de estos riesgos compromete la corrección de lo que el producto sí reporta: afectan a su cobertura, no a su fiabilidad.

11. Oportunidades de mejora

Ordenadas por relación entre valor y esfuerzo:

#	Mejora	Valor	Esfuerzo
1	Aviso de revisión inicial en la primera captura	Alto: cubre el punto ciego principal	Bajo: los datos ya se calculan
2	Deshacer la aceptación de una baseline	Alto	Medio
3	Anti-repetición de notificaciones	Medio: evita saturar al usuario	Muy bajo
4	Implementar o retirar las opciones sin efecto	Medio: credibilidad	Bajo
5	Auditoría automática de dependencias en CI	Medio	Muy bajo
6	Migraciones de esquema de base de datos	Medio: habilita la evolución	Bajo
7	Pruebas de la capa de persistencia	Medio	Medio
8	Firma y notarización de los binarios	Alto para adopción	Alto: requiere cuenta de desarrollador de Apple
9	Publicación del canal de instalación por Homebrew	Medio para adopción	Bajo
10	Modo redactado para compartir reportes	Bajo	Bajo

12. Próximos pasos recomendados

Antes de la siguiente versión

1. Aviso de revisión inicial (oportunidad 1).
2. Deshacer de baseline (oportunidad 2).
3. Cerrar la exposición transitoria de la clave de IA en la lista de procesos.

Corto plazo

1. Anti-repetición de notificaciones e implementación o retirada de las opciones sin efecto.
2. Auditoría de dependencias y migraciones de esquema.
3. Pruebas de persistencia y de comparación contra baseline.

Decisiones que requieren una definición del responsable del producto

1. ¿Se asume el coste de firmar y notarizar? Es el mayor obstáculo de adopción para usuarios no técnicos.
2. ¿Se publica el canal de Homebrew o se mantiene la compilación como vía única?
3. ¿Se mantiene la política de dependencias mínimas aunque impida usar un hash criptográfico para la integridad de la configuración?

13. En una frase

RootCause macOS Inspector es un sensor forense honesto: **no promete detectar amenazas, promete notar lo que cambió y enseñar la prueba** — y esa promesa está respaldada por 112 pruebas automatizadas, cero telemetría verificable en el código y una documentación que declara sus propios límites.

Siguiente lectura recomendada: [18 · Guía para un nuevo desarrollador](#) o, para el detalle técnico, [03 · Arquitectura](#).